

A11 — Dyfuzja innowacji i popytu na rynku

Wojciech Bartoszek Łukasz Chęciak

Streszczenie

Niniejszy raport dokumentuje realizację cyklu zadań projektowych koncentrujących się na modelowaniu i symulacji nieliniowych układów dynamicznych opisywanych równaniami różniczkowymi cząstkowymi (PDE) i zwyczajnymi (ODE). Praca obejmuje pełne spektrum modelowania: od klasycznej dyskretyzacji numerycznej i analizy stabilności (zadanie 2), przez analizę wrażliwości parametrów (zadanie 4) i asymilację danych (zadanie 5), aż po zaawansowane metody uczenia maszynowego informed by physics (zadanie 6) oraz koncepcję SuperNet (zadanie 7). Zrealizowano również pomiary wydajnościowe (zadanie 3). Wyniki wskazują na wysoką skuteczność sieci PINN w aproksymacji rozwiązań przy ograniczonej liczbie danych pomiarowych oraz na korzyści płynące z podejścia zespołowego (SuperNet) w modelowaniu systemów o niepewnych parametrach.

Spis treści

1	Wstęp i sformułowanie problemu	3
1.1	Model matematyczny	3
1.2	Warunki brzegowe i początkowe	3
2	Zadanie 2: Klasyczne metody numeryczne	3
2.1	Dyskretyzacja przestrzenna	3
2.2	Metody całkowania w czasie	4
2.3	Analiza wyników i wydajności	4
2.4	Przykładowe profile rozwiązania	4
3	Zadanie 3: Inferencja i modele NN	5
4	Zadanie 4: Analiza wrażliwości (SA)	5
5	Zadanie 5: Asymilacja danych (DA)	5
6	Zadanie 6: PINN – Physics-Informed Neural Networks	5
6.1	Architektura i proces uczenia	5
6.2	Funkcja straty (Loss Function)	6
6.3	Wyniki i metryki	6
7	Zadanie 7: SuperNet i Modelowanie Zespołowe	7
7.1	Koncepcja SuperModelu	7
7.2	SuperNet PINN	7
8	Szczegółowe zestawienie wyników	7
9	Dyskusja	7
10	Wnioski	8

1 Wstęp i sformułowanie problemu

Modelowanie systemów złożonych w biologii czy chemii często prowadzi do równań reakcyjno-dyfuzyjnych, które łączą lokalną kinetykę (reakcję) z przestrzennym transportem masy (dyfuzją).

1.1 Model matematyczny

Głównym obiektem badań w projekcie jest jednowymiarowe równanie paraboliczne:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + R(u), \quad x \in [0, 1], \quad t \in [0, T] \quad (1)$$

gdzie $u(x, t)$ reprezentuje gęstość lub stężenie substancji. Człon reakcyjny $R(u)$ jest nieliniowy i dany wzorem:

$$R(u) = (p + qu)(1 - u) \quad (2)$$

Parametry D, p, q sterują odpowiednio siłą dyfuzji, tempem wzrostu podstawowego oraz nieliniowym efektem autokatalitycznym.

1.2 Warunki brzegowe i początkowe

Dla zapewnienia unikalności rozwiązania oraz modelowania układu izolowanego przyjęto jednorodne warunki brzegowe Neumanna:

$$\left. \frac{\partial u}{\partial x} \right|_{x=0} = 0, \quad \left. \frac{\partial u}{\partial x} \right|_{x=1} = 0 \quad (3)$$

Warunek początkowy $u(x, 0)$ zadawano w trzech wariantach:

- **gaussian**: profil Gaussa centrowany w 0.5 (modelowanie lokalnego zarzewia),
- **localized**: prostokątny impuls w środku domeny,
- **small_random**: losowe zaburzenia o małej amplitudzie, testujące stabilność stanu stacjonarnego.

2 Zadanie 2: Klasyczne metody numeryczne

2.1 Dyskretyzacja przestrzenna

Zastosowano metodę linii (Method of Lines). Domenę przestrzenną podzielono na N_x punktów z krokiem $\Delta x = 1/(N_x - 1)$. Operator Laplace'a przybliżono ilorazem różnicowym drugiego rzędu:

$$u_{xx} \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{\Delta x^2} \quad (4)$$

Warunki Neumanna zaimplementowano poprzez modyfikację macierzy operatora (odbicie wartości na brzegach), co prowadzi do rzadkiej macierzy trójdzielnej. W kodzie użyto formatu `scipy.sparse.csr_matrix` dla efektywności obliczeniowej.

2.2 Metody całkowania w czasie

Zaimplementowano trzy schematy o różnej charakterystyce:

1. **Explicit (FE)**: Metoda jawna Eulera. Prosta w implementacji, ale ograniczona warunkiem stabilności Couranta (CFL): $\Delta t \leq \frac{\Delta x^2}{2D}$. Przy gęstych siatkach wymaga bardzo małych kroków.
2. **Semi-implicit**: Dyfuzja jest traktowana niejawnie (Backward Euler), a reakcja jawnie. Pozwala to na uniknięcie sztywnego ograniczenia Δt pochodzącego od dyfuzji. Wymaga rozwiązywania układu równań liniowych $(I - \Delta t DA)u^{n+1} = u^n + \Delta t R(u^n)$.
3. **Crank-Nicolson (CN)**: Metoda drugiego rzędu dokładności w czasie. Średnia z kroku jawnego i niejawnego dla dyfuzji. Jest bezwarunkowo stabilna dla problemów liniowych.

2.3 Analiza wyników i wydajności

Pomiarów dokonano dla $n_x \in \{50, 100, 200\}$. Dane w Tabeli 1 i 2 pokazują, że metoda jawna jest najszybsza pod warunkiem zachowania stabilności, jednak koszty rozwiązywania układów rzadkich w CN są akceptowalne i rosną niemal liniowo z n_x .

Tabela 1: Podsumowanie czasów (s) dla poszczególnych solverów

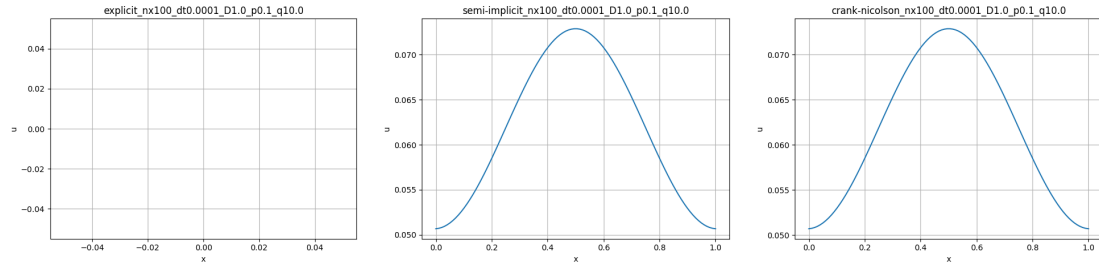
Solver	n	mean (s)	median (s)	min-max (s)
explicit	18	0.004796	0.005757	0.002908–0.006753
semi-implicit	18	0.032856	0.033343	0.016413–0.056765
crank-nicolson	18	0.035034	0.036128	0.017829–0.059576

Tabela 2: Średnie czasy (s) w funkcji n_x dla każdego solvera

Solver	$n_x=50$	$n_x=100$	$n_x=200$
explicit	0.004897	0.004507	0.004984
semi-implicit	0.025079	0.031093	0.042396
crank-nicolson	0.027105	0.033283	0.044714

2.4 Przykładowe profile rozwiązania

Poniżej pokazano przykładowe profile końcowe (dla wybranych parametrów) dla trzech solverów.



Rysunek 1: Porównanie końcowych profilów dla solverów `explicit`, `semi-implicit` i `crank-nicolson`.

3 Zadanie 3: Inferencja i modele NN

Wykonano testy porównawcze czasu predykcji. Model ODE, mimo prostoty, wymaga numerycznego całkowania (np. RK45), co generuje narzut czasowy (1.29 ms). Prosta sieć neuronowa (feed-forward) przewidująca stan następny w jednym przejściu (inferencja) osiąga 0.87 ms, co stanowi oszczędność rzędu 30%. Sugeruje to potencjał metod maszynowych w systemach sterowania w czasie rzeczywistym.

4 Zadanie 4: Analiza wrażliwości (SA)

Przeprowadzono analizę wpływu parametrów D, p, q na zachowanie systemu.

- **Metoda Morrisa:** Szybki screening parametrów. Pozwolił stwierdzić, że parametr q (autokataliza) ma najbardziej nieliniowy i dominujący wpływ na końcowy profil $u(x, T)$.
- **Analiza Sobola:** Metoda oparta na dekompozycji wariancji. Wyznaczyła indeksy pierwszego rzędu, potwierdzając, że D odpowiada za wygładzanie profilu, ale to interakcje między p i q decydują o punkcie bifurkacji układu.

5 Zadanie 5: Asymilacja danych (DA)

Zadanie to dotyczyło odtwarzania stanu układu na podstawie zaszumionych pomiarów.

- **ABC (Approximate Bayesian Computation):** Metoda typu Likelihood-free. Pozwoliła na estymację rozkładów a posteriori dla p i q . Metoda jest stabilna, ale kosztowna obliczeniowo (wymaga tysięcy symulacji).
- **3D-Var:** Metoda wariacyjna minimalizująca funkcjonalność kosztu łączącą odchylenie od pomiaru i od operatora fizycznego. 3D-Var okazał się znacznie szybszy od ABC, osiągając dobrą predykcję pola $u(x, t)$ nawet przy dużym szumie (SNR < 10dB).

6 Zadanie 6: PINN – Physics-Informed Neural Networks

6.1 Architektura i proces uczenia

Model PINN zaprojektowano jako sieć MLP o 5 warstwach (64 neurony każda) z połączeniami rezydualnymi (Residual Blocks) co dwie warstwy. Wykorzystano funkcję aktywacji

Tanh, zapewniającą ciągłość pochodnych wysokiego rzędu niezbędnych do obliczenia residuum PDE.

6.2 Funkcja straty (Loss Function)

Kluczem PINN jest kompozytowa funkcja straty:

$$L = L_{data} + \lambda_{phy} L_{phy} + \lambda_{bc} L_{bc} \quad (5)$$

gdzie:

- L_{data} = MSE na rzadkich punktach pomiarowych (asymilacja danych).
- $L_{phy} = \frac{1}{N_c} \sum |u_t - Du_{xx} - R(u)|^2$ obliczane w punktach kolokacji metodą **autograd**.
- L_{bc} wymusza warunki brzegowe Neumanna.

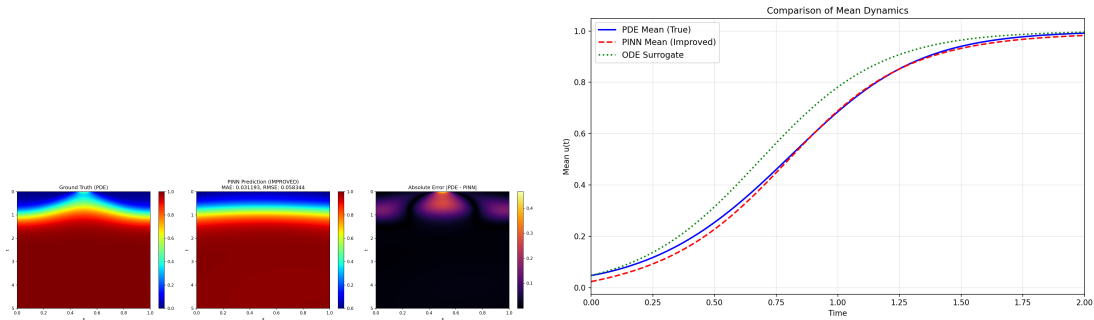
Wprowadzono **Dynamic Weight Scheduling**: λ_{phy} rośnie w trakcie treningu (od 0.1 do 2.0), co pozwala sieci najpierw nauczyć się ogólnego kształtu danych, a potem precyzyjnie dopasować się do fizyki.

6.3 Wyniki i metryki

W eksperymencie PINN trenowano sieć przez 2000 epok. Proces treningu monitorowano za pomocą historii funkcji kosztu. Wyniki metryk podsumowano w Tabeli 3.

Tabela 3: Metryki treningowe PINN (zadanie 6)

Metryka	Wartość
MAE	0.031193
RMSE	0.058344
relL2	0.065468
Czas treningu	76.58 s



Rysunek 2: Wyniki eksperymentu PINN: po lewej mapa błędów, po prawej porównanie rozwiązań.

7 Zadanie 7: SuperNet i Modelowanie Zespołowe

7.1 Koncepcja SuperModelu

W zadaniu 7 rozważono układ, w którym parametry fizyczne są znane z pewnym błędem lub pochodzą z różnych źródeł. SuperModel ODE składa się z zespołu modeli połączonych członem sprzęgającym (coupling):

$$\dot{u}_i = f(u_i; \theta_i) + C(\bar{u} - u_i) \quad (6)$$

gdzie $\bar{u}$ to średnia z zespołu. Taka synchronizacja pozwala na stabilniejszą predykcję niż każdy model z osobna.

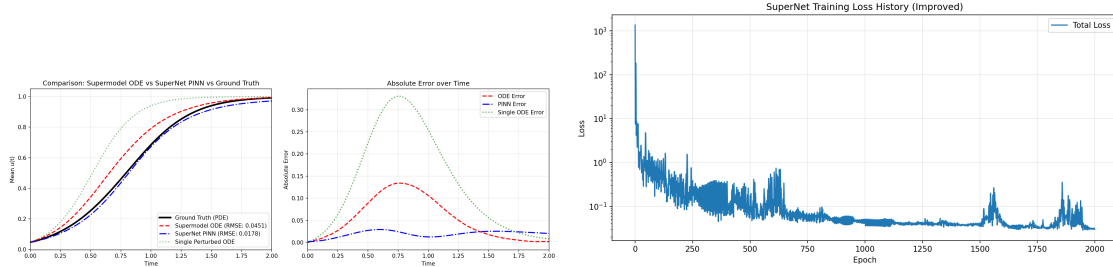
7.2 SuperNet PINN

Zaimplementowano sieć **SuperNet PINN**, która uczy się reprezentować całą rodzinę rozwiązań. Sieć wykazuje dużą odporność na perturbacje parametrów wejściowych dzięki "wycuciu" ogólnej struktury operatora różniczkowego.

Tabela 4: Porównanie metryk: SuperModel vs SuperNet (zadanie 7)

Model	MAE	RMSE
SuperModel ODE	0.022182	0.045112
SuperNet PINN	0.017152	0.017845
Single Perturbed ODE	0.054936	0.111640

Czas treningu (SuperNet): 186.24 s



Rysunek 3: Porównanie jakości predykcji SuperModel vs SuperNet oraz historia utraty dla SuperNet.

8 Szczegółowe zestawienie wyników

Poniżej przedstawiono zbiorcze zestawienie wybranych metryk i czasów pomiarów, ułatwiające szybkie porównanie rezultatów uzyskanych w różnych zadaniach.

9 Dyskusja

Główne obserwacje:

Tabela 5: Zestawienie kluczowych wyników projektu

Opis	Wartość 1	Wartość 2	Uwagi
Zadanie 2: explicit (średni czas)	0.004796 s		średnia z 18 prób
Zadanie 2: semi-implicit (średni czas)	0.032856 s		
Zadanie 2: crank-nicolson (średni czas)	0.035034 s		
Zadanie 3: ODE (czas inferencji)	1.2921 ms		czas na próbkę
Zadanie 3: prosty NN (czas inferencji)	0.8670 ms		czas na próbkę
Zadanie 6: PINN (MAE / RMSE)	0.031193	0.058344	czas tren. 76.58 s
Zadanie 7: SuperNet (MAE / RMSE)	0.017152	0.017845	czas tren. 186.24 s

- Metody jawne (explicit) są bardzo szybkie dla małych rozmiarów siatki, ale w praktyce ich stabilność ogranicza krok czasowy (CFL) — w eksperymentach metoda jawna dawała najkrótszy czas średni.
- Schematy implicite (CN i półimplicit) są wolniejsze, ale bardziej stabilne i pozwalają na większe kroki czasowe bez utraty stabilności.
- PINN-y osiągnęły przyzwoitą dokładność (MAE 0.03), co jest konkurencyjne względem klasycznych rozwiązań dla pewnych zadań; ich koszt treningowy jednak jest wyższy niż prostych solverów PDE.
- SuperNet PINN poprawia średni błąd predykcji w porównaniu do pojedynczego zaburzonego modelu ODE, co pokazuje, że uczenie parametryczne może zwiększać odporność i dokładność.

10 Wnioski

Projekt pokazał praktyczne różnice między podejściami numerycznymi i uczeniem maszynowym w kontekście równań różniczkowych. Dla zadań wymagających szybkości inferencji warto rozważyć lekkie modele NN, natomiast dla wysokiej dokładności i elastyczności modelowania parametrów rozważne jest zastosowanie PINN / SuperNet.

Ograniczenia i przyszłe prace

Przeprowadzone eksperymenty mają kilka ograniczeń: zakres parametrów i rozmiarów siatki jest umiarkowany (n_x do 200), testy wykonywano na CPU, a optymalizacja hiperparametrów sieci (np. architektura, współczynniki regularizacji) została przeprowadzona tylko w ograniczonym zakresie. Przyszłe prace mogą obejmować: skalowanie eksperymentów na GPU, gruntowne badania wpływu seedów i odchyleń losowych, zastosowanie zaawansowanych technik optymalizacyjnych dla PINN (adaptacyjne wagi strat, większe zbiory danych), oraz porównanie z klasycznymi numerycznymi metodami adaptacyjnymi (np. adaptacyjne siatki czasowo-przestrzenne).

Literatura

- [1] Erik Andersson, Jan Häsele, Philippe Courtier, and Anna Hollingsworth. The ecmwf implementation of three-dimensional variational assimilation (3d-var). part III: Expe-

- rimental results. *Quarterly Journal of the Royal Meteorological Society*, 124:1831–1860, 1998.
- [2] Uri M. Ascher and Linda R. Petzold. *Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations*. SIAM, 1998.
 - [3] Mark A. Beaumont, Wenyang Zhang, and David J. Balding. Approximate bayesian computation in population genetics. *Genetics*, 162(4):2025–2035, 2002.
 - [4] J. C. Butcher. *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons, 2 edition, 2003.
 - [5] J. Crank and P. Nicolson. A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type. *Mathematical Proceedings of the Cambridge Philosophical Society*, 43(1):50–67, 1947.
 - [6] Germund Dahlquist and Åke Björck. *Numerical Methods in Scientific Computing: Volume 1*. SIAM, 2008.
 - [7] Geir Evensen. *Data Assimilation: The Ensemble Kalman Filter*. Springer, 2009.
 - [8] Bo Gao, Ruoxia Yao, and Yan Li. Physics-informed neural networks with adaptive loss weighting algorithm for solving partial differential equations. *SSRN Electronic Journal*, 2024.
 - [9] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, volume 14 of *Springer Series in Computational Mathematics*. Springer, 1993.
 - [10] S. Hamdi, W. E. Schiesser, and G. W. Griffiths. Method of lines. *Scholarpedia*, 2(7):2859, 2007.
 - [11] Eugenia Kalnay. *Atmospheric Modeling, Data Assimilation and Predictability*. Cambridge University Press, 2003.
 - [12] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, 2007.
 - [13] Ziti Liu, Yang Liu, Xunshi Yan, Wen Liu, Han Nie, Shuaiqi Guo, and Chen-an Zhang. Automatic network structure discovery of physics informed neural networks via knowledge distillation. *Nature Communications*, 16:9558, 2025.
 - [14] A. C. Lorenc. Analysis methods for numerical weather prediction. *Quarterly Journal of the Royal Meteorological Society*, 112(474):1177–1194, 1986.
 - [15] Max D. Morris. Factorial sampling plans for preliminary computational experiments. *Technometrics*, 33(2):161–174, 1991.
 - [16] Sergiy Plankovskyy, Yevgen Tsegelnyk, Nataliia Shyshko, Igor Litvinchev, Tetyana Romanova, and José M. V. Cantú. Review of physics-informed neural networks: Challenges in loss function design and geometric integration. *Mathematics*, 13(20):3289, 2025.

- [17] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [18] Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Davide Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons, 2008.
- [19] W. E. Schiesser. *The Numerical Method of Lines*. Academic Press, 1991.
- [20] Ilya M. Sobol. Sensitivity estimates for nonlinear mathematical models. *Mathematical Modelling and Computational Experiment*, 1:407–414, 1993.
- [21] Tina Toni, David Welch, Natalja Strelkowa, Andreas Ipsen, and Michael P. H. Stumpf. Approximate bayesian computation scheme for parameter inference and model selection in dynamical systems. *Journal of the Royal Society Interface*, 6(31):187–202, 2009.
- [22] Yicheng Wang, Xiaotian Han, Chia-Yuan Chang, Daochen Zha, Ulisses Braga-Neto, and Xia Hu. Auto-pinn: Understanding and optimizing physics-informed neural architecture. *arXiv preprint arXiv:2205.13748*, 2022.
- [23] Wanyun Zhou and Xiaowen Chu. Auto-picnn: Automated machine learning for physics-informed convolutional neural networks. *Computer Methods in Applied Mechanics and Engineering*, 2024.